Mirage: The Casino Card Game
Liam Wray and Abdul Hasna
Student No: 251230783, 251312802
Group 162

Response to D1 Feedback and Change Log:

The biggest feedback received from Deliverable 1 was to be more precise in our requirements and diagrams, and to make the diagrams visually easier to understand. For example, our requirements were deemed to be too vague so we have specified them and made sure exactly what our Mirage Game is intended to provide for the user. We have also changed the Sequence and UML diagrams to be more reader friendly, communicating clearly how our system is structured.

What has changed since Deliverable 1 requirements:

- Functional Requirements and Traceability: One of the main concerns in Deliverable1 was that our functional requirements needed to be more specific and easily testable. For example, we still have the functional requirements such as FR1: The game must allow the user to choose the game they want to play", however for Deliverable 2 we have made the requirement more precise. The new FR1 is "The game must implement a main menu which appears when the game is built, and display all of the available games. When the user clicks the button of a game, the game appears in a new menu while the main menu disappears".
- The diagrams are more clear and concise. The feedback from Deliverable 1 was that the diagrams were visually hard to understand and not clear, so for Deliverable 2 we made it our focus to visually and easily follow UML diagrams.
- The feedback also indicated that our use case descriptions in Deliverable 1 were missing actors, preconditions and alternative flows. We made sure that they were clearly highlighted in Deliverable 2.
- Another large portion of our feedback was to highlight the difference between the UI and domain logic of the system. To address this issue we specifically separated the UI logic and backend logic using interfaces, and made each game window such as BlackjackWindow and PokerWindow handle game specific interactions with the user. Other classes such as IGame, GameManager and GameWindow reduce the number of duplicated methods and create extensibility for other future classes. Our intent to separate UI and backend logic is also shown more clearly in our UML diagrams, an improvement we have made since Deliverable 1.
- The feedback also included that the purpose of the GameManager class was not defined enough. GameManager is responsible for using the IGame interface to directly manage the games including starting and ending them. This design allows GameManager to

interact with Blackjack, Poker, and Baccarat through shared operations without GameManager knowing the actual game specific logic behind the methods.

- Another change from Deliverable 1 is the addition of newer interface classes and added complexity to the system design. For example we have added a GameFactory, which is in charge of creating games that are specified by the client, handled in a different class. The GameFactory addition also implements the Factory pattern in our system. Using the GameFactory class, the client does not need to know which game is created and how the game is created, creating a loosely coupled relationship.
- We have also increased our focus on the extensibility of the system. In our earlier designs in Deliverable 1 we did not stress the importance of creating a system to allow for new games to be added easily. That is why the classes such GameManager, GameWindow, IGame exist. They all allow for new games and new windows to be implemented easily without substantial code modification.
- The flyweight pattern was implemented. The flyweight pattern allows for cards to be created and used in the game, without taking up lots of memory space. In our old system design cards were created and stored in memory, so every time a game was created the multiple cards used would be taking up unnecessary memory slots. In the current implementation the flyweight pattern allows the game to be less memory intensive.
- We also created animation classes such as QtCardAnimation and IGameAnimation so that the animations can be shared between cards and to decouple animation functionality from the game windows. The animation classes allow BlackjackWindow and PokerWindow to reuse the same card dealing and flipping animations without implementing that logic as part of their own classes, redefining the use of abstraction, polymorphism and DIP into the game.

Why these changes were made: While I was working on the system I noticed I was reusing a lot of the same code for different classes. For example when I was creating each of the games (Poker, Blackjack, Baccarat) I needed a method to start each game, so I called the method startRound(). Then the GameManager could interact with the interface directly without having to know which game it was interacting with. Furthermore I wanted the system to be more extensible, because as I added more games to the project I realized that I would have to go into each individual class and modify the code which was quite time consuming. I changed the game creation logic to be handled by GameFactory and created the GameManager class to manage all the games because I wanted the addition of more games to be implemented easier.

Pattern Justification:

Factory Pattern:

- **Description:** Going into the project and learning about the Factory Pattern, I knew we needed to use this Pattern. The Factory Pattern (a creational pattern) creates an object

depending on user preference. Without the Factory pattern, the user would have to choose which game they would like to play, and also create the object in the same method. This creates tight coupling and if the system needed to add more games it would not be open to extension. Within the game I created the GameFactory class which is in charge of taking the user input from a different class and creating a game object based on the user specification. Otherwise the same class is in charge of both choosing which game to create and prompting the user for their choice. This violates the single responsibility principle and the open-closed principle. This is an issue in my system because without the Factory Pattern the code which specifies which game to create must also know how to construct each game. As the number of games increases, the game creation logic may become scattered instead of being centralized in one class. Furthermore, adding another game requires heavily changing the code that is already in charge of creating games which is time consuming. Using the GameFactory I can streamline the process and ensure the game creation is only happening in one class.

- **Benefits:** Which game is being played and the creation of the game are in two separate classes, allowing for more games to be added more easily in the future. Using a Factory pattern also allows for the game creation logic to be in one class instead of risking the logic being spread across the system.
- **Consequences:** By adding a Factory Pattern there is increased complexity and abstraction. By adding Factory pattern, we introduce more complexity and abstraction with the introduction of a new class. Additionally, when adding new games, there still needs to be some modification to the GameFactory class.
- **Alternate Options:**
    - Not using a Factory Pattern: If I didn't use a Factory Pattern the game creation logic would be handled by each of the specific game classes, but this would make adding new games much more complicated and time consuming.
    - Abstract Factory: Supports groups of related objects which could have worked instead of a normal Factory pattern, but it would have been unnecessarily complex to implement.

Implementation: GameFactory.cpp, GameFactory.h, IGame.h, GameManager.h

Flyweight Pattern:

- **Description:** The flyweight pattern was the structural design pattern we chose to implement as a part of Deliverable 2. It was implemented to reduce the unnecessary duplication of the Card (and consequently Deck objects) across multiple casino games. Previously every time a new game was created a Deck object and 52 new cards would be created, which increases overhead. Using the flyweight pattern, I created a CardFactory class, which is responsible for creating the Card objects and sharing them with all of the created games. Each game references the created Card objects by using the CardPtr. Each

game has its own Deck, but the Deck references the shared created Card objects instead of creating new Cards for each game.

- **Consequences and Costs: of the Flyweight pattern:** Using the flyweight pattern each game does not have to create their own Deck objects every time they are created, they just point to one Deck class and share its intrinsic properties. The flyweight pattern is implemented in the CardFactory class, where the CardFactory shares the same Card objects to create a Deck, and the 52 cards created for the Deck are then shared between all of the games. Furthermore, the CardFactory becomes solely responsible for creating and using Card objects, and prevents a modification of a card object throughout each game. The cost is that there is added complexity into the system, and harder to debug.
- **Alternate Options:**
    - Not implementing the flyweight pattern: This would be simpler and logically easier to follow, but this would mean each game would create their own Decks and Cards so when many games are played overhead increases.
    - Card creation inside of UI: I could have also made the UI handle the card creation but this violates the Single Responsibility Principle.
- **Implementation:** Card.h, Card.cpp, CardFactory.cpp, CardFactory.h, Deck.h, Deck.cpp, Player.h

SOLID Principles:

Single Responsibility Principle:

- **Description:** Each class is focused on completing one task. For example the Player class is only in charge of what attributes the Player has in a real life context and the actions a Player does, such as having a hand. Instead of the player knowing which game they are a part of or the score of the game, that logic is handled by the specific game they're playing such as Blackjack or Poker. Another example is implementing the GameManager class. The GameManager class is responsible for starting and stopping the current game, whereas before the individual game windows (blackjackwindow and pokerwindow) were responsible for not only displaying a functional frontend with buttons and labels, but were also in charge of managing the game. I took that responsibility and created GameManager, so that each class now only has only one responsibility.
- **Before and After:** Before when the classes had multiple responsibilities, debugging and adding new features was complex and the project was beginning to turn into "spaghetti code". WIth SRP implemented each class has only 1 responsibility, showing debugging and abstraction of classes becomes much easier.

Dependency Inversion Principle (DIP):

- **Description:** The dependency inversion principle is used to decouple classes from each other, so that adding new methods or structures to one class does not require the change

of another class. We applied the DIP through the GameManager class and IGame interface, making it so the GameManager can work with different games through the IGame interface. This approach reduces the dependency classes have with each other, and creates a system where new games can be added more easily.

- **Before and After:** Before DIP was implemented the code the classes were tightly coupled together, so implementing new classes required lots of existing code to constantly be changing to accommodate for new extensions. With DIP dependencies can be injected easily and the correct methods can be called based on the injected dependency, allowing for looser coupling and easier extension.

Open-Closed principle:

- **Description:** The open closed principle is important to define how the system is structured to maximize extensibility. The system should be open for extension, closed for modification. This is demonstrated in the Factory Pattern, as new games can be added to the Factory, and the Factory can extend to create them. Without having to modify the existing classes, I can easily create more games and create minimal additions to other classes to integrate new games, without having to extensively go through each class and change the necessary items. Furthermore, the Open-Closed Principle is shown in the GameWindow class, where PokerWindow and BlackjackWindow inherit methods from the GameWindow class without knowing what the methods specifically do. This also allows for new games to be added without the existing code being modified, such as a new Baccarat game. The Baccarat game just has implemented the methods specified in GameWindow, allowing the system to be open for extension but closed for modification.
- **Before and After:** Without using the open closed principle the system was tightly coupled, where adding one new class or interface required many other classes to be changed. To implement multiple games with multiple interfaces we knew that the system needed to be open for extension and closed for modification, so we designed the system beforehand to follow the Open-Closed principle.

Current SOLID Violation: Our system design prioritized being easily extensible and easy to add new classes, the current implementation of GameFactory does not fully follow the Open Closed Principle. When a new game is created, the GameFactory must be modified for a new game to be added. Therefore other classes must be modified for new classes to be added, breaking the Open Closed Principle. For

Challenges and Resolutions:

The challenges when creating Deliverable 2 was conceptually thinking about the game for each and every detail, no matter how big or small. When the number of classes started to increase as more games and windows were created I needed a way of making sure that all of the logic was

still connected to each other. For example, it was quite difficult when thinking how the GameManager would manage an object created by a GameFactory which has to implement IGame and the correct game is started for whichever game is created. As you can imagine, this got quite confusing quite quickly. The way I found a solution to this problem was to write and draw all of the class relationships beforehand. Not so much as a UML class diagram, just to visualize the thought process and to understand exactly what each method needed to do. So for the Hit function, I wrote out a step by step process of how the Hit function will be implemented as part of every class. When the player clicks the Hit button Qt emits a clicked signal, the Blackjack window invokes the onHit() method. The onHit method executes the actual hit method in Blackjack, which is how the UI communicates with the backend logic. The hit method returns a CardPtr, which then the BlackjackWindow uses in conjunction with IGameAnimation to display and animate.

Plans for Deliverable 3:

Our core functionality for the game revolves around showcasing why going to a casino is statistically not in your favour, as the odds are in favour of the dealer/house. Within Deliverable 2 we have created classes that mimic a traditional casino, as players can choose which games to play (Poker, Blackjack). For Deliverable 3 we plan to implement statistics for each game as well Player statistics. Our planned features are to show win/loss statistics, game specific calculations (eg: winning percentage of each hand), as well as statistical explanations to explain the calculation methods and assumptions. To implement these plans for Deliverable 3 we will be utilizing the Builder pattern to construct different game specific stats when they are requested.

For the testing strategy we plan to use Google Test and Google Mock to test the functionality of the system, we intend to use Google Test to test individual classes, for example we test whether the Deck class correctly deals cards and if each game calculates scores aligning with the rules. As for Google Mock, the plan is to test the GameManager and GameWindow classes. We will test GameManager by creating a mock implementation of the IDame interface, to verify that the correct game methods are called. As for testing GameWindow, we could create a mock implementation of the IGameAnimation in order of verifying that the correct animation methods are called without needing to run the actual graphical animations.

When necessary, we will modify our classes so that these dependencies are allowed to be injected. With this procedure we would be able to test individual components independently and verify that the different classes interact correctly.